MCP-Lock: Lockfiles and Pre-Launch Gating for Coding-Agent MCP Servers

Aakash G S
Dept. of Data Science and Business Systems
SRM Institute of Science and Technology
Chennai, India
gsaakash05@gmail.com

Hariharan J
Dept. of Data Science and Business Systems
SRM Institute of Science and Technology
Chennai, India
harinithe1831@gmail.com

J Jaswanth Raj
Dept. of Data Science and Business Systems
SRM Institute of Science and Technology
Chennai, India
jjjashjjj@gmail.com

***Abstract*—Coding agents start Model Context Protocol (MCP) servers with unpinned commands such as `npx -y pkg`, so each start may run a version whose code, and whose tool descriptions that the model reads as instructions, nobody has reviewed. A census of 2,000 MCP packages finds that 36.5% changed their advertised tools within 90 days, and that 97.0% of these changes carry no provenance, publisher or install-script signal. We present MCP-Lock, a lockfile that pins each server's archive, build identity, publisher, install scripts and tool definitions, and a gate that checks every launch against it. In a pre-registered evaluation, MCP-Lock detects twice as many real attacks as the closest prior gate (39.8% vs. 19.6% of 490), almost entirely through install-script diffs (96 of 99 added detections). Almost all its false blocks come from the baseline's age rule; exempting locked servers from that rule cuts them from 33.4% to 0.73%. An adaptive attacker still gets past the gate in 7 of 12 attacks; five exploratory hardening switches close all but two, and we measure what each costs on 8,180 benign upgrades.**

***Index Terms*—software supply chain, Model Context Protocol, coding agents, provenance, lockfiles, pre-registration**

I. Introduction

An MCP server gives a language-model agent its tools. Coding-agent harnesses start most of them from package registries with a runner command: `npx -y @scope/server`, `uvx server`. Kapner et al. found unpinned MCP declarations in 9.8% of 2,660 agent setups [1]. An unpinned launch resolves to whatever version is latest at that moment, so the code that runs, and the tool descriptions the model reads as instructions, can change between two sessions without any user action. Tool descriptions are an injection channel [2], and a server that changes them after it has been approved is the "rug pull" of the MCP threat literature [2], [3].

Existing pre-install checks do not look at this layer. The hook of Bagmar and Saraf [4] checks names, sources and age before `pip install`; registry provenance (SLSA [5], Sigstore [6], PEP 740 [7]) says who built an archive, not what tools it advertises; and the public MCP registry keeps no versioned record of tool definitions [8]. Scanners and proxies that pin the live tool list after a server starts exist [9]–[11]; none of them reports how often honest servers change tools, or what pinning catches and costs on real packages.

We ask three questions. **RQ1:** how often do MCP servers change their tool definitions, and do metadata signals reveal those changes? **RQ2:** does pinning provenance and tool definitions catch more real attacks than the existing gate, and at what cost? **RQ3:** what does an attacker who knows every check do, and what would stop them?

Contributions. (i) A census of MCP packages on npm and PyPI: metadata for all 15,272 eligible packages and statically extracted tool definitions for a stratified sample of 2,000 (Section III). (ii) MCP-Lock, a lockfile format and a gate that never executes package code (Section IV). (iii) A pre-registered evaluation against the closest prior gate on real malicious versions, synthetic rug pulls and benign upgrades, with a kill rule fixed in advance (Section V). (iv) An adaptive-attacker study with five hardening switches (D1–D5), each reported with its cost on benign upgrades (Section VI). Code, data and the registration are public (Section X).

II. Background and Threat Model

MCP. A server exposes tools; the client sends `tools/list` and receives, for each tool, a name, a description and an input schema, which it places in the model's context [12]. The list may change at run time, signalled by `notifications/tools/list_changed`. Security surveys catalogue tool poisoning, rug pulls and malicious servers [3], [13].

Registry provenance. npm can attach a SLSA provenance attestation, signed through Sigstore, that names the source repository, workflow and builder of a release [5], [6], [14]; PyPI offers the same through PEP 740 [7]. npm also records the publishing account and lifecycle scripts, which run at install time. Hijacked maintainer accounts and malicious install scripts are recurring attack patterns [15]–[18]; the 2025 Shai-Hulud worm combined both [19].

Threat model. A developer runs a coding agent with the MCP-Lock hook and a lockfile created while each server was benign. The attacker wants their code to run, or their text to reach the model, through a later launch. We consider five tiers: *T1* a squatter with a new package name; *T2* a thief of one maintainer's registry token; *T3* an attacker inside the release pipeline (workflow or merged commit); *T4* the package owner (malicious or sold); *T5* the owner of a dependency. Registry compromise, Sigstore forgery and attacks on the model itself are out of scope.

III. RQ1: Tool-Definition Drift

Frame and sample. We built a frame of MCP packages from the official MCP registry and from MCP configuration files on GitHub, and kept packages with at least two versions that resolve on their registry: 15,272 packages. From these we drew a stratified random sample of 2,000 (by ecosystem and download tercile; fixed seed).

Extraction. For every selected version we fetched the archive, verified its registry digest, and extracted tool definitions statically from the source (registration calls in JavaScript/TypeScript, decorators and registrations in Python). Values that cannot be resolved without execution become an opaque marker, so a rebuild of unchanged source keeps its hash. The canonical hash is SHA-256 over the sorted, canonically serialised (name, description, input schema) triples. On 82 held-out servers that we started in a network-less container and queried with `tools/list`, tool-name precision was 92.2% and recall 78.0%: detected changes are real, and drift figures are lower bounds.

Results. 36.5% [34.5, 38.7] of sampled packages (731/2,000) changed their tool definitions within a 90-day window; among packages that shipped any release in the window, 59.0% [56.2, 61.7] did. Across release transitions with extractable tools on both sides, 33.7% (2,259/6,695) changed tools: 987 added a tool, 132 removed one, 1,374 changed a description and 1,157 an input schema. Tool changes are routine.

Tool changes are also invisible to supply-chain metadata. 97.0% [96.3, 97.7] of tool-changing transitions had no provenance loss, publisher change, maintainer change or new install script on the same release, and none of these signals was more frequent on tool-changing than on other releases (Fisher's exact test per signal, four tests, all $p \geq 0.25$). Over twelve months of all 228,044 releases in the frame, 3.65% of active npm packages lost build provenance at least once, 2.25% went from CI publishing to a human account, and 1.90% added an install script; only 25.9% (npm) and 46.4% (PyPI) of latest versions carry provenance. Because tool changes are frequent and independent of these rare metadata signals, a gate can only see them by looking at the tools.

IV. MCP-Lock Design

Lockfile. `mcp-lock.json` sits next to the harness configuration. For each server it pins the exact version, the archive digest (npm SRI or PyPI SHA-256), the build identity (repository, workflow, builder) when provenance exists, the npm publisher and maintainers, the install scripts, and the canonical tool hash with the tool names. The continuity identity deliberately excludes ref and commit, which change on every honest release. A clean upgrade is a regenerated entry and a reviewable diff.

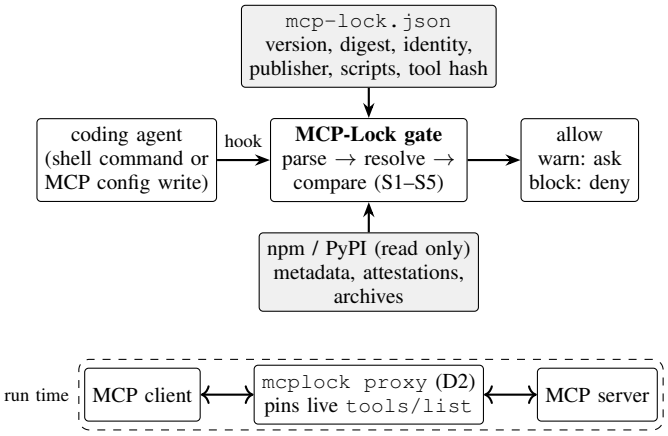


Fig. 1. MCP-Lock. Before launch, the gate compares what a command would fetch with the lock and never runs package code. At run time, the optional proxy (hardening D2) checks every tool listing against a pin.

TABLE I
Gate signals. S2–S5 compare a launch against its lock entry.

Signal	Fires when	Action
S1	name confusable, absent or younger than 30 days; untrusted source; hidden index; known-vulnerable pin	block
S2	provenance disappears, or a different repository / workflow / builder built the new version	block
S3	npm: published by hand after CI releases, or by a non-maintainer	warn
S4	new install-time lifecycle script (PyPI: sdist-only release after wheels)	block
S5	canonical tool hash differs (diff shown)	warn
gate	server not locked / same version, different digest	warn / block

Gate. Before a command runs, the gate parses it (installs and runner launches, including `npx`, `pnpm dlx`, `bunx`, `uvx`, `pipx run`), resolves what the launch would fetch now, and compares it with the lock entry. It reads registry metadata, attestations and archives, and never executes package code. Fig. 1 shows the flow and Table I lists the signals; S1 is our reimplementation of the Bagmar and Saraf checks [4]. A block denies the command; a warning asks the user. The gate runs as a Claude Code `PreToolUse` hook on shell commands and on writes to MCP configuration files, and fails closed to "ask" on internal errors.

V. RQ2: Pre-Registered Evaluation

Design. Hypotheses, corpora, scoring and a kill rule were registered on OSF [20] before any scored run, with the gate frozen at a tagged commit. **H1:** on real attacks, G (S1–S5) detects more cases than S1-C, the stronger reading of the

TABLE II
CONFIRMATORY RESULTS (WILSON 95% CIS).

	S1-C	G
Detection on R	19.6% [16.3, 23.3]	39.8% [35.6, 44.2]
block only	19.6% [16.3, 23.3]	39.2% [35.0, 43.6]
False blocks on B	32.9% [31.9, 34.0]	33.4% [32.3, 34.4]
Warnings on B	0	16.5% [15.7, 17.3]
H1	99 G-only vs. 0 S1-C-only cases; exact McNemar $p = 3.2 \times 10^{-30}$ (Holm 6.3×10^{-30})	
H2	false blocks +0.42 pp [0.28, 0.56]; $p = 1.2 \times 10^{-10}$	
Kill rule	99 added detections ≥ 2: not killed	

baseline. **H2:** on benign upgrades, G's false-block rate differs from S1-C's. **Kill rule:** the method claim is withdrawn if S2–S5 add fewer than two detections while G's false-block rate is equal or worse. Detection is block or warn; H1 and H2 use exact McNemar tests [21] with Holm correction [22]; proportions carry Wilson 95% intervals [23]. Six deviations, all logged before the affected results, concern replay mechanics and corpus construction.

Corpora. R-B: 22 scenarios of the baseline paper. R-M: 468 real malicious npm/PyPI versions from OpenSSF and Datadog reports [24], [25], 368 hijacked existing packages (class C, locked at the last clean version) and 100 new or typosquatted ones (class N). R = R-B ∪ R-M, $n = 490$. R-S: synthetic rug pulls on a local registry (four classes × 10 servers, 10 controls), reported separately. B: 8,180 benign upgrades (6,695 MCP, the census transitions of Section III; 992 top npm; 493 top PyPI). Every case is replayed offline as of its publication time; malicious archives are only read, never executed.

Results. Table II gives the confirmatory results. H1 holds: G detects more real attacks. H2 holds in the unfavourable direction: G's false-block rate is 0.42 percentage points higher than S1-C's, a small but significant cost. The kill rule is not triggered. Counting only blocks, which do not depend on a user reading a warning, barely changes detection (39.2% vs. 19.6%). In the ablation, removing S4 loses 96 of G's detections, removing S5 2, S3 1 and S2 0. On hijacked packages (class C) G detects 31.5% [27.0, 36.4] against 4.6% for S1-C, mostly install scripts added by the 2025 worm campaigns [19], [26]. By publication date, 32 and 30 of the 99 added detections fall in the two Shai-Hulud waves (14–30 Sep. and 21 Nov.–5 Dec. 2025). Outside both windows the gain remains: G detects 32.0% of 344 R-M cases against 21.2% for S1-C (37 vs. 0 discordant cases, $p = 1.5 \times 10^{-11}$; exploratory). On new names (class N) both detect 59.0% through the baseline's age rule. Provenance continuity (S2) added nothing on real attacks because removed malicious versions leave no registry record, so S2 and S3 were evaluable on only 88 of 468 R-M cases, and only 13.6% of class-C locks had provenance to begin with.

On the synthetic rug pulls, G detects 40/40 and S1-C 0/40; S5 alone carries the two tool-definition classes, S4 the install-script class, and S2 and S3 each catch the stolen-token class.

TABLE III
EVASION MATRIX. G: FROZEN GATE; D: ALL HARDENING SWITCHES D1–D5. E11 COUNTS DETECTED COMMAND SHAPES.

Case	Tier	Attack	G	D
E1	T1	new name, edit distance ≥ 2, older than 30 days	warn	warn
E2	T2	stolen token; lock has no provenance	allow	block
E3	T2	stolen token; lock has provenance	block	block
E4	T3	payload via the genuine release workflow	allow	allow
E5	T4	poisoned description built at run time	warn	block
E6a/b	T4	tools fetched from a URL; new / no new version	allow	block
E7	T4	poison in an input-schema field description	warn	block
E8a/b	T4	time-bomb tool / tool from a base64 blob	warn/allow	block
E9	T5	payload and tools in a caret-range dependency	allow	block
E10	T2/T4	extra file added to a locked PyPI release	allow	block
E11	any	22 command shapes (detected)	13/22	21/22

No control was flagged (0/10).

G's cost is extra reviews. Its false blocks barely move (34 more than S1-C), but S5 adds warnings on 16.5% of benign upgrades, essentially all in the MCP stratum, where honest servers change tools often (RQ1). Per added detection, G costs 0.34 extra false blocks and 13.6 extra warnings. The baseline's own 32.9% false-block rate comes from its 30-day age rule replayed at upgrade time (40.1% of MCP upgrades). Median gate latency is 0.99 s on a cold cache (p95 3.19 s; $n = 199$) and 0.05 s warm.

Age-rule sensitivity (exploratory). Almost all of G's false blocks come from S1's 30-day age rule, which guards the first use of a name but fires again on every upgrade of a young server that the user has already locked. Waiving it for locked servers, and changing nothing else, lowers G's false blocks on B from 33.4% to 0.73% [0.57, 0.94]. Interruptions (block or warn) fall from 49.8% to 28.4%; warnings rise to 27.7%, because some former blocks also carried a tool-drift warning, which now surfaces on its own. Detection on R drops by 8 hijacked packages that were young enough for the age rule to catch, to 38.2% (187/490), still 91 more than S1-C. This was not pre-registered; the confirmatory results above are unchanged.

VI. RQ3: ADAPTIVE ATTACKER

Setup. For each tier we built the cheapest attack we could find against a gate whose checks, lock format and parser are known, as 12 packages on a local registry with inert payloads, plus 22 command-line variants of one attack (Table III). All runs use the frozen gate; nothing was published publicly or executed.

What evades. The frozen gate blocks 1 and warns on 4 of the 12 attacks; 7 pass. Static extraction cannot see tools built at run time or fetched from a URL (E6, E8b), and a

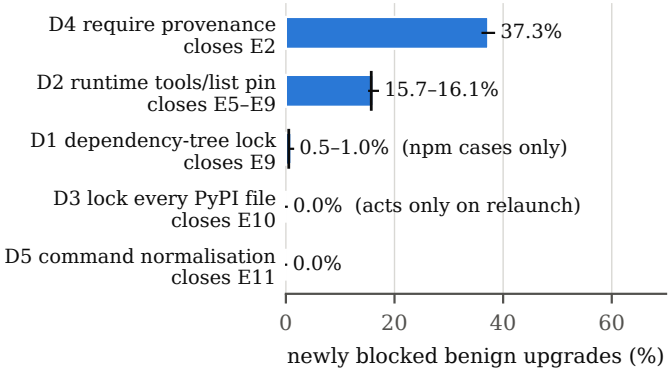


Fig. 2. Cost of each hardening switch: benign upgrades it would newly block on top of the frozen gate (bar: upper estimate with Wilson 95% CI; tick: lower estimate). D1 is measured on the 5,652 npm upgrades, D3 on PyPI.

rug pull through a remote file needs no new version at all (E6b). Locking the top-level archive leaves dependencies free to move (E9). A stolen token defeats S2 and S3 whenever the lock never had provenance (E2), and the same-version check compares only the locked file (E10). Commands hidden behind `npm exec`, `sh -c` or a shell variable escape the parser. The conditions these attacks need are common: 60.1% of benign locks have no provenance (E2), 21.7% of versions hide their tools from static extraction (E6/E8), and 93.9% of npm servers float their MCP SDK dependency (E9, a lower bound that counts one dependency). A payload shipped through the genuine release pipeline (E4) carries genuine provenance; no lock can tell it apart from an honest release.

Hardening (exploratory). We implemented five switches, off by default; with all off the gate is the frozen code, and a re-replay of every Phase-4 corpus is identical case by case. *D1* locks the resolved npm dependency tree and applies the S2–S4 rules to every dependency whose version changed. *D2* is a stdio proxy that pins the live `tools/list` per tool (trust on first use), rejects listings that add or change a tool, and refuses calls to unpinned tools; it checks every listing, so a time bomb is caught when it fires; this is the design of existing runtime pinning tools [10], [11], measured here. *D3* locks every file of a PyPI release. *D4* requires provenance. *D5* normalises the evasive command shapes. With all five, 10 of 12 attacks block and 21 of 22 command shapes are caught; E1 still only warns and E4 passes.

Cost. Figure 2 shows the cost of each switch. D3 and D5 cost nothing on benign upgrades: D5 changes no decision (0/8,180), and D3 only acts when the locked version is relaunched, which no upgrade does; as an exposure measure, 0.47% of locked PyPI releases later gained a file. D1 adds blocks to 0.53%–1.03% of npm upgrades, depending on whether the lock was taken just before the upgrade or when the old version was released. A strict tree lock that blocks every dependency change, as `npm ci` would, blocks 12.3%–48.9% instead; applying continuity rules to dependencies, instead of pinning them exactly, is what keeps the tree lock usable. D2 and D4 are expensive: D2 newly blocks 15.7%–16.1% of benign upgrades, all MCP servers that changed tools, and D4 37.3%. The live tool list of a historical version cannot be observed without running it, so D2's benign cost assumes it equals the static extraction, and the 1,483 upgrades without statically visible tools are counted as costless.

Safer alternatives (exploratory). A block tells the agent what not to run but not what to run instead, so an agent may retry with something worse. When the gate flags a command, it now proposes a replacement and shows it only if the gate itself passes the rewritten command; the decision is never changed. For a locked server it suggests the newest version between the lock and the target that passes every lock check, falling back to the locked version; for a typosquat, the popular package the name imitates; for a vulnerable pin, the lowest release with no advisory, stepping forward when a fix carries advisories of its own. The Claude Code hook appends the suggestion to its deny reason, so the agent can retry with it. A verified suggestion is offered for 114/116 real hijacked versions (class C), 40/40 synthetic rug pulls and 17/18 flagged adaptive attacks. Every one of them is the locked version, and none is the malicious version (0/114 for class C, 95% CI [0.0, 3.3]). For R-B it names the intended package in 20/20 typosquat scenarios and a fixed release for 4/4 vulnerable pins. It stays silent on all 59 newly registered malicious names, none of which is within edit distance 2 of a popular package. Class C correctness holds by construction, since the lock is the last version no advisory lists. On benign upgrades the suggestion is always a rollback to the reviewed version (4,075/4,075), which is safe but defers an honest update until the user re-locks. Suggesting a typosquat's target or a fixed version is existing practice (e.g. `npm audit fix`); here every suggestion, including the rollback, is checked by the gate before the agent sees it.

VII. Discussion and Limitations

What the evidence supports. Pinning catches real attacks that the name-and-source checks miss, but the confirmed gain is an install-script diff (S4), a classic supply-chain signal rather than an MCP-specific one. A deployable gate should not re-apply first-use checks to servers already in the lock: doing so is what made both gates block a third of honest upgrades. Tool pinning (S5) is what catches MCP rug pulls, and it is necessary because metadata signals do not accompany tool changes (RQ1), yet it warns on 19.7% of honest MCP upgrades. Of these 1,319 warnings, 46.9% change a description, 38.8% add a tool, 13.4% change only an input schema and 0.9% only remove tools (exploratory). The first three are what a rug pull changes, so S5 cannot be narrowed without losing the attacks it exists for. The 2 real attacks that S5 alone detects both report removed tools; no case in R is a known tool-definition rug pull, so the evidence that S5 stops rug pulls comes from R-S and the adaptive cases. Provenance continuity is sound but rarely applicable: most packages have none, and attackers who remove their versions take the evidence with them.

Simpler alternatives (exploratory). Two steps need no gate. Pinning the exact version in the MCP configuration prevents every class-C case (368/468 of R-M) but holds back

all 8,180 benign upgrades until the user edits the configuration. npm's `--ignore-scripts` keeps the install-time code of up to 26.6% of npm R-M cases from running (an upper bound, since a payload may also run at launch) and skips an honest install script in 2.6% of npm upgrades. Together with the baseline hook it covers 42.2% of npm R-M cases, about as many as G (41.9%), so on real attacks most of G's gain is available more cheaply. What G adds is continuity of tool definitions and provenance, which the real attacks barely exercised, unattended upgrades when nothing changed, and a diff to review when something did.

Limits of a lock. Every lock in this paper trusts the state at lock time and flags change. An attacker in the release pipeline (T3) produces change that is indistinguishable from an honest release, and a runtime pin can only compare against what was trusted first. Reviewing the diff remains the user's job; the gate makes the diff visible.

Threats to validity. Static extraction under-counts tools (recall 78.0%), so drift and S5 coverage are lower bounds. R-M contains only attacks that were caught and reported; attacks still undetected are absent by definition. Replay reconstructs metadata as of each case's time but cannot recover removed versions, which is why S2/S3 evaluability is low. Class C follows a mechanical rule (an earlier clean version exists) and may include attacker-owned packages with a benign first release. The adaptive attacks were built by the authors of the defence; the hardening results are exploratory and were not pre-registered, as are the safer-alternative suggestions and the publication-window split. D2's matrix verdicts use live tool lists written down from each attack's source; running the 11 locked servers in containers with no outside network gave the same D2 verdict in 11/11 cases and the same findings in 11/11, with E8a's clock set past its trigger date. Time of lock is unknown in practice, which is why D1 is reported as a range.

VIII. Related Work

MCP security. Surveys and taxonomies describe tool poisoning, rug pulls and malicious servers [3], [13], and practitioners described the rug-pull pattern early on [2]. Hasan et al. analyse the security and maintainability of open-source servers [27]. Closest to RQ1, Kraishan measures drift of the fields the MCP registry versions (descriptions, packages, endpoints) across 8,900 multi-version servers and notes that tool definitions are not versioned there [8]; we measure the tool definitions themselves from package archives, and build the client-side pinning that work recommends. Metere proposes signed admission and per-server tool allowlists for remote servers [28], and ETDI signed, versioned tool definitions [29]; MCP-Lock instead needs no change to servers or registries.

Tool pinning. Pinning tool definitions is established practice. MCP-Scan hashes tool descriptions to detect rug pulls [9], and open-source proxies pin each tool's definition in a committed lockfile and block a changed listing; one also records the package version and registry digest [10], [11]. These tools start the server to read its tools. MCP-Lock's S5 reads them from the archive before launch, combines them with provenance, publisher and install-script continuity, and is evaluated on real attacks and benign upgrades; our runtime proxy (D2) follows the existing design.

Agent supply chains. Bagmar and Saraf weaponise setup instructions against coding agents and propose the pre-install hook we use as baseline [4]. Kapner et al. scan agent configurations and report unpinned MCP declarations [1]. Shan finds in a pre-registered audit that coding assistants almost never check trust signals before installing [30], which motivates a gate that checks on their behalf.

Package supply chains. Attack catalogues [15], [16] and ecosystem studies [17], [18] document account takeover and install-script abuse, the patterns behind S3 and S4. Sigstore [6], SLSA [5] and PEP 740 [7] make build identity verifiable; MCP-Lock uses that identity for continuity rather than for absolute trust.

IX. Conclusion

MCP servers change the tools they present to models often, and nothing in their metadata says so. A lock that pins archive, provenance, publisher, install scripts and tool definitions catches 99 real attacks that the existing gate misses, at a cost paid mostly in reviews of honest tool changes; on npm, the existing gate with `--ignore-scripts` covers about as many. Against an adaptive attacker, the frozen gate blocks 1 and warns on 4 of 12 attacks; dependency-tree locking, runtime tool pinning, file-level PyPI locking and command normalisation close most of the rest, and we measure what each costs. Two gaps remain open for any lock-based defence: attacks shipped through the legitimate release pipeline, and the review fatigue of honest change.

X. Artifact Availability

Code, lockfile specification, replay bundles, results and logs: https://github.com/ftaakash/MCP-Lock. Pre-registration: https://osf.io/2amj3, DOI 10.17605/OSF.IO/2AMJ3. Every number in this paper is generated from the result files by `paper/make_numbers.py`.

References

[1] B. Kapner, C. Soceanu, A. Petrunin, and H. Gartner, "Scanning the harness: Configuration exposures in AI coding-agent supply chains," arXiv:2609.07360v3, 2026.

[2] L. Beurer-Kellner and M. Fischer, "MCP security notification: Tool poisoning attacks," Invariant Labs blog, https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks, Apr. 2025.

[3] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," arXiv:2503.23278, 2025.

[4] A. Bagmar and P. Saraf, "Setup complete, now you are compromised: Weaponizing setup instructions against AI coding agents," arXiv:2607.15143, 2026.

[5] SLSA, "SLSA provenance, version 1.0 (predicate type `https://slsa.dev/provenance/v1`)," https://slsa.dev/spec/v1.0/provenance, 2023.

[6] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 2353–2367.

[7] W. Woodruff, F. Tuesca, and D. Ingram, "PEP 740 – index support for digital attestations," https://peps.python.org/pep-0740/, 2024.

[8] O. Kraishan, "Same name, different server: A security census of silent drift in the Model Context Protocol ecosystem," arXiv:2609.14119, 2026.

[9] L. Beurer-Kellner and M. Fischer, "Introducing MCP-Scan: Protecting MCP with Invariant," Invariant Labs blog, https://invariantlabs.ai/blog/introducing-mcp-scan, Apr. 2025.

[10] GautamTalksDev, "mcp-pin: Pin what your agent approved; block it when it changes," https://github.com/GautamTalksDev/mcp-pin, accessed 20 Oct. 2026, 2026.

[11] confuseddude, "mcpseal: Pins the hash of every approved MCP tool definition," https://github.com/confuseddude/mcpseal, accessed 20 Oct. 2026, 2026.

[12] Model Context Protocol, "Specification (revision 2025-06-18): Tools," https://modelcontextprotocol.io/specification/2025-06-18/server/tools, 2025.

[13] W. Zhao, J. Liu, B. Ruan, S. Li, and Z. Liang, "When MCP servers attack: Taxonomy, feasibility, and mitigation," arXiv:2509.24272, 2025.

[14] npm, "Generating provenance statements," npm Docs, https://docs.npmjs.com/generating-provenance-statements, 2026.

[15] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proc. DIMVA*, ser. LNCS, 2020, pp. 23–43.

[16] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 1509–1526.

[17] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" in *Proc. 44th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022, pp. 331–340.

[18] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proc. 28th USENIX Security Symposium*, 2019, pp. 995–1010.

[19] CISA, "Widespread supply chain compromise impacting npm ecosystem," Alert, https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem, Sep. 2025.

[20] A. G S, "MCP-Lock: Does provenance and tool-definition pinning catch MCP-server supply-chain attacks that name/source checks miss, and at what false-block cost?" OSF Registries, preregistration, Oct. 2026.

[21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, pp. 153–157, 1947.

[22] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian J. Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[23] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[24] OpenSSF, "Malicious packages: Reports of malicious packages in open source repositories (OSV format)," https://github.com/ossf/malicious-packages, 2026.

[25] Datadog, "Malicious software packages dataset," https://github.com/DataDog/malicious-software-packages-dataset, 2026.

[26] A. Bleih, "Shai-Hulud 2.0: Inside the second coming, the most aggressive NPM supply chain attack of 2025," Check Point blog, https://blog.checkpoint.com/research/shai-hulud-2-0-inside-the-second-coming-the-most-aggressive-npm-supply-chain-attack-of-2025/, Nov. 2025.

[27] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, "Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers," arXiv:2506.13538, 2025.

[28] A. Metere, "Attested tool-server admission: A security extension to the Model Context Protocol," arXiv:2605.24248, 2026.

[29] M. Bhatt, V. S. Narajala, and I. Habler, "ETDI: Mitigating tool squatting and rug pull attacks in Model Context Protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control," arXiv:2506.01333, 2025.

[30] P. Shan, "Do AI coding assistants check before they install? A preregistered demand-side audit of trust signals in the research software supply chain," arXiv:2609.07754, 2026.